

NeuraX 3.0 — Pre-Development Analysis

Visual Inspection & Defect Root-Cause Assistant

Domain 2: AI in Industry and Automation

1. Problem Understanding

The organizers are **not** asking for vision AI. They are asking for a **causal decision-support system** for a factory. The vision component is just the *entry point* into a larger reasoning chain:

```
Defect observed (pixels) -> Process condition responsible -> Flow constraint
-> Throughput/loss impact -> Profitability impact -> Decision / recommendation
```

Three things the PS explicitly rejects:

If we build...	Why it fails
A basic image classifier	Answers "defect/no-defect" but cannot say <i>where</i> , <i>why</i> , or <i>what to do</i> . Zero process/economics linkage.
A generic defect detector	Localizes boxes but stops there. No root cause, no bottleneck, no money.
A generic KPI dashboard	Shows OEE/scrap/throughput as disconnected charts. No traceability from a specific defect back to a specific process cause and forward to cost.

What they actually want: a single system where a judge can click one suspicious unit and follow an **unbroken evidence chain** — image -> defect family + location + confidence -> correlated process parameters -> the station that is the bottleneck -> throughput delta -> \$ impact -> recommended intervention. The word "unified" in the PS is the whole assignment.

Two more signals that matter:

- **"Handle unfamiliar/novel defect types instead of blindly forcing them into a known class"** -> they explicitly reward anomaly/OOD handling. This is a differentiator most teams will miss (they'll softmax everything).
- **"Keep recommendations and profitability estimates simulated/advisory"** -> this is a *permission slip*, not a limitation. We should lean into a **digital-twin / what-if simulator**. It also lowers liability and technical risk.

Domain: manufacturing (they mention mixed product variants, batch drift, stations, cycle time, WIP, changeovers, scrap, rework — classic **Theory of Constraints / lean** vocabulary).

2. Evaluation Strategy (feature -> marks mapping)

Criterion	Marks	Feature that earns it	How we prove it to judges
Defect detection & classification	15	Unit-level accept/reject + defect-family classification	Confusion matrix, per-class F1 shown live; overlay label on image
Defect localization	10	Bounding box + heatmap (Grad-CAM-like or anomaly map)	Visual region highlighted on the inspected part
Robustness to unseen conditions	10	Anomaly/OOD detector + "NOVEL DEFECT" state + drift monitor	Demo a held-out/perturbed image -> system flags unknown, not a wrong class
False-reject / false-accept handling	5	Cost-sensitive thresholding + confidence bands + "needs human review" queue	Show threshold slider moving FAR/FRR; show review queue
Root-cause quality	5	Ranked causes linking defect features <-> process params <-> stations	Evidence table with correlation strengths, not a single guessed cause

Criterion	Marks	Feature that earns it	How we prove it to judges
Explainability & confidence	5	Confidence scores, uncertainty, evidence panel, decision trace	Every output accompanied by "why"
Technical implementation	5	Clean modular pipeline, schema-agnostic adapter, E2E runnable	README + architecture diagram + working demo
UI/UX & visualization	5	Premium industrial control-center UI	Distinct look, clear decision chain
Checkpoint 1 (15)		README: Problem Understanding (5) + Architecture (5) + Approach (5)	Written + diagram
Checkpoint 2 (25)		Partial execution: Features + relevance	A running system even if incomplete

Strategic insight on mark distribution: the 60-mark pile is dominated by *vision* (detection 15 + localization 10 + robustness 10 = 35 marks). But the *differentiators* (root cause, explainability, false-accept handling = 15) are where generic teams lose points, and they're cheap for us to win because they're mostly reasoning + UI, not model training.

Checkpoint timing: Checkpoint 1 is pure documentation — we can bank 15 marks early with a strong README + architecture diagram before deep coding. Checkpoint 2 (25) is "does it run and is it relevant" — our MVP should be demoable as early as possible, even with synthetic data.

3. Minimum Viable Product

MUST HAVE (the decision chain, end-to-end)

1. Ingest organizer datasets via a **config-driven adapter** (no hardcoded schema).
2. Unit inspection: accept / defective.
3. Defect classification into known families.
4. Localization: bounding box + anomaly heatmap.
5. **Novel/unknown defect flag** (anomaly score, not forced softmax).
6. Confidence per prediction + "human review" state for low confidence.
7. Production-flow analysis: throughput, cycle time, WIP, utilization per station.
8. **Bottleneck detection** (ranked station constraints).
9. **Throughput & cost impact** estimate of the bottleneck (simple digital-twin math).
10. **Root-cause ranking**: defect family <-> process parameters.
11. Evidence-based recommendation text.
12. UI that presents the full chain on one decision screen.

SHOULD HAVE

- Cost-sensitive threshold tuning (false accept/reject tradeoff UI).
- Batch-to-batch **drift indicator**.
- What-if profitability sliders (changeover, downtime, defect rate).

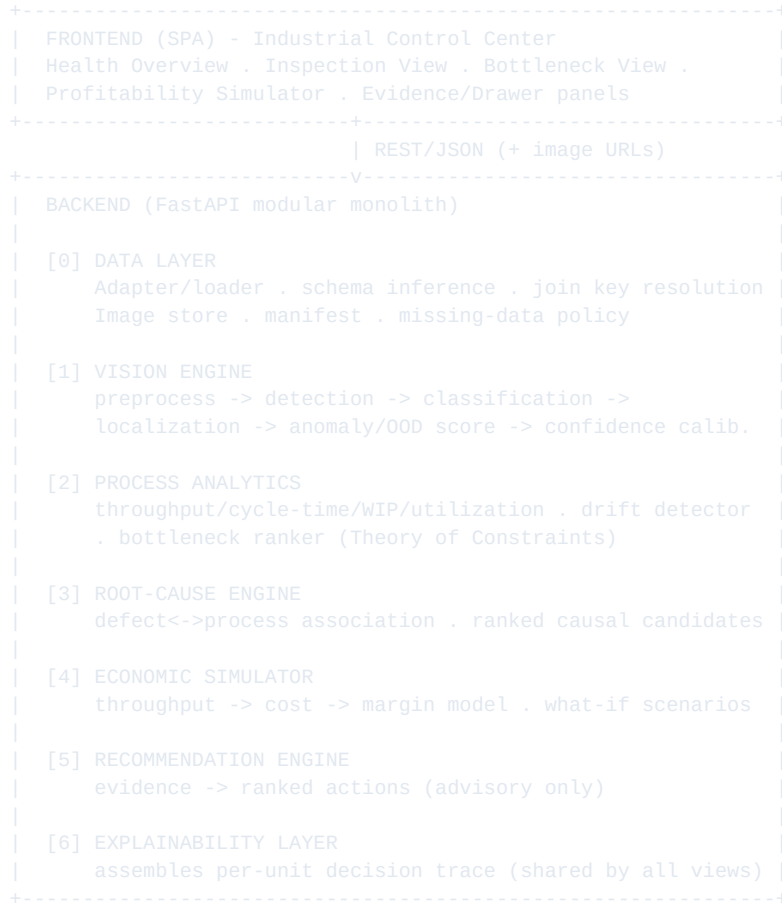
OPTIONAL (only if time remains)

- 3D/Animated production-line visualization.
- Live synthetic simulation mode ("incoming units" streaming).
- Scenario save/compare.

Explicitly NOT building: authentication, multi-user, database clustering, Kafka, microservices, model training from scratch, PLC integrations, anything enterprise.

4. Proposed System Architecture

Single deployable backend + single-page frontend. **No Kafka, no Kubernetes, no queues.** A modular monolith is correct here.



Design rule: everything flows through a single **"Inspection Record"** object (unit id, batch, station, image, prediction, defect family, anomaly score, confidence, features, linked process values). The UI, root-cause, and simulator all read this one object — that's what makes it feel *unified* instead of three glued tools.

No database required — load datasets into memory (pandas) at startup; persist only user-created scenarios to a JSON file. With hackathon dataset sizes this is fine.

5. AI/ML Strategy (simplest effective choices)

> Guiding principle: **use pretrained backbones and classical statistics, train nothing heavy from scratch.** A 24-hour budget cannot train ResNets or LSTMs reliably. None of the marks require deep custom models.

Task	Recommended approach	Why it fits 24h + zero cost
Detection (accept/defect)	Classical CV descriptors (edges, texture, blob stats) -> scikit-learn classifier (RandomForest / GradientBoosting) OR pretrained CNN embeddings (torchvision MobileNet, frozen) -> logistic regression	No training from scratch; works on small data; CPU-friendly; interpretable features
Classification (defect family)	Same pipeline, multiclass. Fallback if organizer gives labeled images: fine-tune only final layer of a frozen backbone	Minutes to train, no GPU needed
Localization	Anomaly heatmap (reconstruction error or patch-distance) + contour/blob detection (OpenCV); OR Grad-CAM on the frozen CNN	Gives <i>region</i> without object-detection training; Grad-CAM is ~20 lines
Novel/unseen defect	Anomaly detection: PCA reconstruction error / Isolation Forest / Mahalanobis distance on embeddings. If score high but class confidence low -> label "NOVEL"	This is the <i>correct</i> way to satisfy the PS. Deterministic, fast, no labels needed

Task	Recommended approach	Why it fits 24h + zero cost
Uncertainty/confidence	Softmax (calibrated) + anomaly score + distance-to-training-distribution. Bands: auto-pass / review / reject	Cheap, and directly earns explainability + false-accept marks
Root cause	Associational reasoning: mutual information / correlation / decision-tree feature importance between defect families and process parameters per station/batch; plus "defects spike when param X moved" change-point check	Statistics, not black box -> easier to explain to judges
Bottleneck detection	Theory of Constraints heuristics on cycle time, utilization, WIP accumulation, downtime, changeover — rank stations by constraint severity	Standard industrial method; transparent formula judges can trust
Profitability estimation	Deterministic cost model (defect cost + throughput loss + downtime + changeover) with scenario parameters	A simulator, not an ML model — auditable, adjustable in UI

Deliberately avoided: LSTMs (no temporal sequence guarantee, expensive, unexplainable), custom object detectors (need annotation + GPU), LLMs for root cause (non-deterministic, can't defend in judging, and cost/latency risk).

Optional zero-cost LLM polish: only for *wording* recommendations, if a free local model is available — never for the actual reasoning.

6. Dataset Strategy

The organizers provide **inspection + production + economic** datasets. We must **not** assume their schema. Build for uncertainty.

Expected (hypothesized, not assumed):

- *Inspection*: unit images and/or image-derived measurements, defect labels, timestamps, unit/batch IDs.
- *Production*: station-level cycle times, quantities, downtime, changeovers, WIP, variants, shifts.
- *Economic*: unit cost, price/margin, scrap/rework costs, downtime cost, throughput value.

On receipt, inspect in this order:

1. File formats, row counts, column names/dtypes, null rates, value ranges.
2. Identify **shared keys**: `unit_id`, `batch_id`, `station_id`, `product_variant`, `timestamp`. This is the *join decision* and the riskiest step.
3. Identify the **label column(s)** in inspection data (is it binary? family? free text?).
4. Identify image location/format if any (folder, path column, embedding table).
5. Identify the **granularity mismatch** (images per unit? multiple units per batch?).

Connection strategy (the "unified" wiring):

- Join inspection -> production on `unit_id/batch_id/timestamp` (nearest-time or batch rollup).
- Join production -> economic on `station_id/variant/period`.
- Build **three denormalized views** for the engines: per-unit, per-station, per-batch.

Preprocessing:

- Timestamp normalization, unit conversions, variant encoding.
- Image: resize, normalize, optional augmentation for the "unseen conditions" demo (brightness/rotation/noise) — this is how we *manufacture* a robustness test if the dataset lacks one.
- Feature engineering for process analytics (cycle-time percentiles, rolling utilization).

Missing/incomplete data policy:

- Never crash. Adapter returns nulls + a **data-quality score** surfaced in UI.
- Impute with station/variant medians, or drop with a visible "excluded N units" note.
- If a join key is missing, degrade gracefully: analyze what's joinable, label the rest "unlinked" in the UI.

If image data is absent or unusable: fall back to a **synthetic image generator** (OpenCV: normal surface + injected scratches/dents/contamination/discoloration on a generated texture) so the vision demo still works. This also gives us a controlled way to demonstrate novel-defect detection. Keep it clearly labeled as simulation.

7. Core Demo Loop (3–5 min judge script)

A single narrative, one screen, one thread pulled all the way through:

1. **Opening frame**: "Line health" — throughput, defect rate, margin, one station glowing red. (10s)
2. **Open the spiking station**. System shows defect rate rising on batch #B-7 since a changeover. (20s)
3. **Inspect a flagged unit**. Image opens; **defect box + heatmap** highlight a scratch region. Label: "Surface scratch, family S1, confidence 0.91." (30s)
4. **Show a second unit the model has never seen**. System refuses to force a class -> **"NOVEL DEFECT — anomaly score 0.86."** (30s) — this is the wow moment for robustness marks.
5. **Pull the evidence chain**. A trace panel: this defect family co-occurs with `temperature up`, `speed down`, on Station 4 since batch B-7; correlation strength shown, alternatives ranked. (45s)
6. **Bottleneck view**. Station 4 flagged as constraint: highest cycle-time variance + WIP pile-up after changeovers. (30s)

7. **Impact numbers.** Removing the constraint -> throughput +X%, defect cost -\$Y, margin +Z%. Show the causal arithmetic. (40s)
8. **Recommendation + what-if.** "Reduce changeover on Station 4 / reverify temperature setpoint." Drag a what-if slider -> profitability curve updates live. (40s)
9. **Close:** false-accept/review queue — "34 units auto-passed, 6 quarantined for review." (20s)

The story is the causal chain, not the accuracy number. End on the chain.

8. UI/UX Strategy

Aesthetic target: an **industrial mission-control / SCADA-modern** look — dark, dense, high-contrast, data-forward. Explicitly *not* a white admin dashboard with sidebar + cards + Material components.

Layout (single app, 3–4 focused views, not 12 tabs):

- **Command Overview:** line topology (stations as nodes with live health color), top-line KPIs, active alerts, margin gauge.
- **Inspection View:** large part image with **click-to-inspect** overlays (box, heatmap, confidence, novel badge).
- **Decision Drawer (the spine):** slide-in panel that, for any selected unit or alert, renders the full chain — evidence -> root cause -> bottleneck -> impact -> recommendation. This drawer *is* the product.
- **Simulator View:** what-if sliders (changeover time, defect rate, downtime) -> live throughput/cost/profit curves.

Visual language:

- Semantic color only: green (nominal), amber (review), red (constraint/defect), violet (novel/unknown).
- Confidence shown as meter/band, never a bare number.
- Every AI claim has a **"why" affordance** (tooltip/expand) — this is how we visibly earn explainability marks.
- Subtle motion (Framer Motion) for state changes and the decision drawer; no gratuitous animation.

Free resources only:

- React + Vite + **Tailwind CSS**; **shadcn/ui** (free, code-owned components) for primitives — style it heavily so it doesn't look default.
- Charts: **Recharts** or **Apache ECharts** (both free/MIT).
- Icons: **Lucide** (free). Motion: **Framer Motion** (free).
- Heatmaps/overlays: render server-side with **OpenCV** as PNG, or client-side canvas.
- Fonts: a free industrial/technical sans (e.g., Inter / IBM Plex).

No ThemeForest, no paid UI kits, no premium chart libraries.

9. 24-Hour Development Plan

Prioritize strictly by marks: vision chain (35) > process/root-cause/explain (25) > economics/UI (10).

Phase	Hours	Deliverable
P0 — Setup & Checkpoint 1	0–2	Repo, README with Problem Understanding + Architecture diagram + Approach (banks 15). Skeleton backend/frontend running.
P1 — Data layer	2–5	Adapter, schema inference, joins, three denormalized views. Or synthetic generator if no data yet.
P2 — Vision engine (core marks)	5–10	Detection + classification + confidence + novel flag + localization heatmap.
P3 — Process analytics	10–13	Throughput/cycle/WIP/utilization + bottleneck ranker. -> Checkpoint 2 demoable here.
P4 — Root cause + economics	13–16	Association engine + cost/profit simulator + recommendation text.

Phase	Hours	Deliverable
P5 — UI decision chain	16–20	The four views + decision drawer + overlays + simulator sliders.
P6 — Polish & demo rehearsal	20–22	False-accept/review queue, drift indicator, motion, rehearsal of the script.
P7 — Buffer	22–24	Fixes, README final, backup recording of demo.

Abandon-first list if time runs short:

1. 3D visualization -> 2D topology.
2. Live streaming simulation -> static batch.
3. Drift detector -> last.
4. Scenario save/compare -> drop.
5. Fancy motion -> drop to static transitions.

Never cut: the novel-defect state, localization overlay, bottleneck->cost arithmetic, and the decision drawer. Those carry the marks.

10. Risks & Fallbacks

Risk	Impact	Fallback
Dataset arrives late / unusable	Blocks everything	Build against synthetic generator first ; adapter swaps in real data later. Keep engines schema-agnostic.
No image data / images are embeddings only	Vision marks at risk	Use synthetic images for demo; if embeddings exist, classify/anomaly-score embeddings and mock the visual overlay.
No shared join keys between datasets	"Unified" story breaks	Join on time/batch heuristics; if impossible, present datasets as parallel evidence with an explicit linkage assumption note.
Labels are messy/free-text/imbalanced	Low accuracy	Normalize labels, fall back to binary accept/defect, use anomaly score as primary signal.
CNN unavailable / install issues offline	Vision stalls	Pure OpenCV + scikit-learn path must exist from day one (no torch dependency in the critical path).
Localization weak	Loses 10 marks	Heatmap via classical anomaly map (patch difference from a learned "good" template) — always works.
Over-engineering eats the clock	No demo	Enforce the modular monolith; timebox each engine to a stub-then-improve pattern.
Judge asks "is this real?"	Credibility	Label simulated/advisory outputs explicitly; that's what the PS asks for.

11. Final Architecture Recommendation

One stack. Build this.

- **Backend:** Python 3, **FastAPI** (single modular monolith), pandas/numpy for data, **scikit-learn** for detection/classification/root-cause stats, **OpenCV** for preprocessing + anomaly heatmap + synthetic image generation, **PyTorch (CPU, torchvision)** *optional* for frozen-embedding features — but never in the critical path. In-memory data, JSON for scenarios.
- **Frontend:** **React + Vite + TypeScript + Tailwind CSS + shadcn/ui**, **Recharts** (or ECharts), **Framer Motion**, **Lucide** icons. Single SPA, four views + one decision drawer.
- **Communication:** plain REST/JSON; images served as static files. No queues, no DB, no Docker required (optional single-container for demo safety).
- **Intelligence model:** frozen-embedding + classical ML + anomaly scoring for vision; correlation/decision-tree association for root cause; Theory-of-Constraints heuristics for bottlenecks; deterministic cost model for profitability.

- **Cost:** \$0. All components free/open-source; data simulated if needed.

Why this wins: it satisfies the 35 vision marks with proven cheap techniques, wins the differentiating 15 reasoning/explainability marks with transparent statistics, banks Checkpoint 1 early via documentation, gets Checkpoint 2 with a runnable MVP by hour ~13, and the decision-drawer UI makes the *unified causal chain* — the actual ask — legible to judges in under five minutes.

Blocking question before we start: Do we already have the organizer datasets (or at least their schema), or should Phase P1 begin by building the synthetic data generator so we can proceed unblocked? This decision only affects ordering, not the architecture.